Retail Promotion Effectiveness

Diego Alejandro Ross Price

Technical appendix: data extraction, ETL, quality assurance, and SQL analytical workflow

TECHNICAL APPENDIX · R, PYTHON, POSTGRESQL, SQL

This appendix documents the technical workflow used to build the retail promotion effectiveness analysis. It covers data extraction from the Complete Journey package, exploratory data analysis, quality validation, relational loading into PostgreSQL, and the reusable SQL queries that supported campaign, category, household, and coupon-level evaluation.

Technical appendix overview

The implementation was organized as an end-to-end analytical pipeline rather than as isolated scripts. The workflow begins with source extraction in R, moves through Python-based preparation and exploratory data analysis, loads curated tables into PostgreSQL, and finishes with a reusable SQL layer designed for business-facing analysis and dashboard consumption.

The appendix keeps the implementation close to the original project while presenting it in a cleaner technical format. Explanatory notes focus on intent, modeling choices, and data-engineering decisions so the code can be reviewed as part of a coherent analytical deliverable.

> The SQL layer was designed around a consolidated analytical view so campaign, promotion, coupon, and household analyses could be executed from a consistent transactional base with reduced query duplication.

Stage	Primary objective	Core tools
Source extraction	Export the Complete Journey tables into interoperable flat files for downstream work.	R · completejourney · readr
Preparation and EDA	Validate schema quality, standardize identifiers, profile data ranges, and inspect categorical consistency.	Python · pandas
Relational loading	Load curated tables into PostgreSQL and create indexes for analytical performance.	Python · SQLAlchemy · PostgreSQL
Analytical SQL	Create reusable SQL assets for campaign, category, coupon, and household analysis.	PostgreSQL · SQL
Reporting readiness	Prepare a stable analytical layer for executive reporting and dashboard integration.	SQL · BI-ready output

Data acquisition and preparation

Data extraction

The source data was accessed in R because the Complete Journey package exposes the integrated tables natively in that environment. The extraction step exported both the package-included datasets and the downloaded full transaction and promotion tables into CSV format so the rest of the workflow could be executed in a database-oriented pipeline.

R EXTRACTION SCRIPT

```
# ==========================================
# COMPLETE JOURNEY -> EXPORT ALL DATASETS TO CSV
# ==========================================

# 1) Install required packages if needed
required_packages <- c("completejourney", "readr")

to_install <- required_packages[!required_packages %in% installed.packages()[, "Package"]]
if (length(to_install) > 0) {
  install.packages(to_install)
}

# 2) Load packages
library(completejourney)
library(readr)

# 3) Create output folder
output_dir <- "complete_journey_csv"
if (!dir.exists(output_dir)) {
  dir.create(output_dir, recursive = TRUE)
```

```
}

# 4) Load datasets included in the package
datasets_included <- list(
  campaigns = campaigns,
  campaign_descriptions = campaign_descriptions,
  coupons = coupons,
  coupon_redemptions = coupon_redemptions,
  demographics = demographics,
  products = products,
  promotions_sample = promotions_sample,
  transactions_sample = transactions_sample
)

# 5) Export included datasets to CSV
for (dataset_name in names(datasets_included)) {
  write_csv(datasets_included[[dataset_name]],
            file.path(output_dir, paste0(dataset_name, ".csv")))
  cat("Exported:", dataset_name, "\n")
}

# 6) Download full datasets (promotions and transactions)
full_data <- get_data(which = "both", verbose = TRUE)

# 7) Export full datasets if download was successful
if (!is.null(full_data)) {
  promotions_full <- full_data[[1]]
  transactions_full <- full_data[[2]]

  write_csv(promotions_full, file.path(output_dir, "promotions_full.csv"))
  cat("Exported: promotions_full\n")

  write_csv(transactions_full, file.path(output_dir, "transactions_full.csv"))
  cat("Exported: transactions_full\n")
} else {
  cat("The full datasets could not be downloaded.\n")
}

cat("Done. All CSV files were saved in:", normalizePath(output_dir), "\n")
```

Data preparation and quality assurance

Python was used for exploratory data analysis, schema validation, and field normalization before database loading. The preparation stage inspected table structure, checked whether identifier fields stored as floats contained meaningful decimals, converted temporal fields, and profiled numeric, date, and categorical columns for range, coverage, duplicate-pattern, and missing-value review.

The profiling step was intentionally explicit. Rather than applying blind type coercion, the process checked whether fields such as product_id and basket_id behaved like true integers before standardization. A structured profiling dictionary was also used to keep numeric, temporal, and categorical review reproducible across tables.

PYTHON PREPARATION AND EDA

```
import pandas as pd

# Show all columns during inspection
pd.set_option("display.max_columns", None)

# Load source tables
campaigns_df = pd.read_csv(r"Data/campaigns.csv")
campaign_descriptions = pd.read_csv(r"Data/campaign_descriptions.csv")
products = pd.read_csv(r"Data/products.csv")
coupons = pd.read_csv(r"Data/coupons.csv")
coupon_redemptions = pd.read_csv(r"Data/coupon_redemptions.csv")
promotions_sample = pd.read_csv(r"Data/promotions_sample.csv")
demographics = pd.read_csv(r"Data/demographics.csv")
transactions_sample = pd.read_csv(r"Data/transactions_sample.csv")

# Inspect sample records
print(campaigns_df.head(10))
print(campaign_descriptions.head(10))
print(products.head(10))
```

```
print(coupons.head(10))
print(coupon_redemptions.head(10))
print(promotions_sample.head(10))
print(demographics.head(10))
print(transactions_sample.head(10))

# Review structure and data types
print(campaigns_df.info())
print(campaign_descriptions.info())
print(products.info())
print(coupons.info())
print(coupon_redemptions.info())
print(promotions_sample.info())
print(demographics.info())
print(transactions_sample.info())

# Check whether float IDs contain real decimals
print(((products["product_id"].dropna() % 1) != 0).sum())
print(((coupons["product_id"].dropna() % 1) != 0).sum())
print(((transactions_sample["basket_id"].dropna() % 1) != 0).sum())

# Standardize ID fields as integers
products["product_id"] = products["product_id"].astype(int)
coupons["product_id"] = coupons["product_id"].astype(int)
transactions_sample["basket_id"] = transactions_sample["basket_id"].astype(int)

# Convert date and datetime fields
campaign_descriptions["start_date"] = pd.to_datetime(
    campaign_descriptions["start_date"],
    format="%Y-%m-%d",
    errors="coerce"
)
campaign_descriptions["end_date"] = pd.to_datetime(
    campaign_descriptions["end_date"],
    format="%Y-%m-%d",
    errors="coerce"
)
coupon_redemptions["redemption_date"] = pd.to_datetime(
    coupon_redemptions["redemption_date"],
    format="%Y-%m-%d",
    errors="coerce"
)
transactions_sample["transaction_timestamp"] = pd.to_datetime(
    transactions_sample["transaction_timestamp"],
    format="%Y-%m-%dT%H:%M:%SZ",
    errors="coerce",
    utc=True
)

# Define numeric profiling fields
numeric_checks = {
    "campaign_descriptions": ["campaign_id"],
    "products": ["product_id", "manufacturer_id"],
    "coupons": ["coupon_upc", "product_id", "campaign_id"],
    "coupon_redemptions": ["household_id", "coupon_upc", "campaign_id"],
    "promotions_sample": ["product_id", "store_id", "week"],
    "demographics": ["household_id"],
    "transactions_sample": [
        "household_id", "store_id", "basket_id", "product_id", "quantity",
        "sales_value", "retail_disc", "coupon_disc", "coupon_match_disc", "week"
    ]
}

# Define temporal profiling fields
date_checks = {
    "campaign_descriptions": ["start_date", "end_date"],
    "coupon_redemptions": ["redemption_date"],
    "transactions_sample": ["transaction_timestamp"]
}

# Define categorical profiling fields
category_checks = {
    "campaign_descriptions": ["campaign_type"],
    "products": ["department", "brand", "product_category", "product_type", "package_size"],
```

```
    "promotions_sample": ["display_location", "mailer_location"],
    "demographics": [
        "age", "income", "home_ownership", "marital_status",
        "household_size", "household_comp", "kids_count"
    ]
}

# Profile numeric variables for ranges and outliers
for df_name, cols in numeric_checks.items():
    print(f"\n{'='*20} {df_name.upper()} | NUMERIC {'='*20}")
    print(globals()[df_name][cols].describe().T)

# Profile date fields for temporal coverage
for df_name, cols in date_checks.items():
    print(f"\n{'='*20} {df_name.upper()} | DATES {'='*20}")
    df = globals()[df_name][cols]
    print(df.agg(["count", "min", "max"]).T)

# Profile categories for duplicates and missing values
for df_name, cols in category_checks.items():
    print(f"\n{'='*20} {df_name.upper()} | CATEGORIES {'='*20}")
    df = globals()[df_name]
    for col in cols:
        s = df[col]
        s_norm = s.astype(str).str.strip().str.lower().replace({"nan": pd.NA})
        print(f"\n{col}")
        print("unique_raw:", s.nunique(dropna=False))
        print("unique_normalized:", s_norm.nunique(dropna=False))
        print("nulls:", s.isna().sum())
        print(s.value_counts(dropna=False).head(15))
```

PostgreSQL data loading

Once the tables were validated in Python, the cleaned DataFrames were loaded into PostgreSQL as replaceable analytical tables. This step established the relational environment used for downstream SQL development and added indexes on the primary join and filtering keys to improve performance across household, campaign, coupon, product, store, and week-level analysis.

PYTHON TO POSTGRESQL LOADING

```
from sqlalchemy import create_engine, text

# Define PostgreSQL connection settings
user = "<postgres_user>"
password = "<postgres_password>"
host = "localhost"
port = "5432"
database = "retail_promo"

# Create a database engine to connect Python with PostgreSQL
engine = create_engine(f"postgresql+psycopg://{user}:{password}@{host}:{port}/{database}")

# Group all cleaned DataFrames for export
tables = {
    "campaigns": campaigns_df,
    "campaign_descriptions": campaign_descriptions,
    "products": products,
    "coupons": coupons,
    "coupon_redemptions": coupon_redemptions,
    "promotions_sample": promotions_sample,
    "demographics": demographics,
    "transactions_sample": transactions_sample
}

# Load each DataFrame into PostgreSQL as a table
for table_name, df in tables.items():
    df.to_sql(
        name=table_name,
        con=engine,
        if_exists="replace",
        index=False,
        method="multi",
        chunksize=5000
```

```
    )

# Create indexes to improve join and filter performance
with engine.begin() as conn:
    conn.execute(text("CREATE INDEX IF NOT EXISTS idx_campaigns_household_id ON campaigns(household_id)"))
    conn.execute(text("CREATE INDEX IF NOT EXISTS idx_campaigns_campaign_id ON campaigns(campaign_id)"))
    conn.execute(text("CREATE INDEX IF NOT EXISTS idx_campaign_descriptions_campaign_id ON
campaign_descriptions(campaign_id)"))
    conn.execute(text("CREATE INDEX IF NOT EXISTS idx_products_product_id ON products(product_id)"))
    conn.execute(text("CREATE INDEX IF NOT EXISTS idx_coupons_product_id ON coupons(product_id)"))
    conn.execute(text("CREATE INDEX IF NOT EXISTS idx_coupons_campaign_id ON coupons(campaign_id)"))
    conn.execute(text("CREATE INDEX IF NOT EXISTS idx_coupon_redemptions_household_id ON
coupon_redemptions(household_id)"))
    conn.execute(text("CREATE INDEX IF NOT EXISTS idx_coupon_redemptions_coupon_upc ON
coupon_redemptions(coupon_upc)"))
    conn.execute(text("CREATE INDEX IF NOT EXISTS idx_coupon_redemptions_campaign_id ON
coupon_redemptions(campaign_id)"))
    conn.execute(text("CREATE INDEX IF NOT EXISTS idx_promotions_product_store_week ON
promotions_sample(product_id, store_id, week)"))
    conn.execute(text("CREATE INDEX IF NOT EXISTS idx_transactions_household_id ON
transactions_sample(household_id)"))
    conn.execute(text("CREATE INDEX IF NOT EXISTS idx_transactions_product_id ON
transactions_sample(product_id)"))
    conn.execute(text("CREATE INDEX IF NOT EXISTS idx_transactions_store_id ON
transactions_sample(store_id)"))
    conn.execute(text("CREATE INDEX IF NOT EXISTS idx_transactions_week ON transactions_sample(week)"))
    conn.execute(text("CREATE INDEX IF NOT EXISTS idx_transactions_basket_id ON
transactions_sample(basket_id)"))
```

SQL analysis

The SQL layer was organized around one reusable analytical view plus a set of business-facing queries. Each query addresses a specific analytical question while reusing common transactional logic, product descriptors, demographic enrichment, and promotion flags. This structure reduced repetitive joins and supported more consistent KPI definitions across the project.

Module	Business purpose
vw_txn_enriched	Create a consolidated transactional layer with product, demographic, and promotion context.
Initiative performance	Compare campaigns, promotion placements, and redeemed coupons under a common KPI structure.
Category responsiveness	Estimate directional lift for promoted versus non-promoted categories.
Household segmentation	Evaluate response heterogeneity by income, age, and household composition.
Incrementality and coupon flows	Assess campaign lift, exposure cohorts, opportunity areas, and coupon follow-up behavior.

Reusable analytical view

A reusable SQL view was created to assemble the analytical grain used throughout the project. The view combines transaction records with promotion placements, product attributes, and household descriptors while also creating convenience fields such as transaction_date, total_discount_value, and a binary promotional exposure flag.

SQL VIEW · VW_TXN_ENRICHED

```
CREATE OR REPLACE VIEW vw_txn_enriched AS
SELECT
    t.household_id,
    t.store_id,
    t.basket_id,
    t.product_id,
    t.week,
    t.transaction_timestamp,
    t.transaction_timestamp::date AS transaction_date,
```

```
    t.quantity,
    t.sales_value,
    COALESCE(t.retail_disc, 0) AS retail_disc,
    COALESCE(t.coupon_disc, 0) AS coupon_disc,
    COALESCE(t.coupon_match_disc, 0) AS coupon_match_disc,
    COALESCE(t.retail_disc, 0)
      + COALESCE(t.coupon_disc, 0)
      + COALESCE(t.coupon_match_disc, 0) AS total_discount_value,
    CASE WHEN pr.product_id IS NOT NULL THEN 1 ELSE 0 END AS promo_exposed,
    COALESCE(pr.display_location, 'No Display') AS display_location,
    COALESCE(pr.mailer_location, 'No Mailer') AS mailer_location,
    p.department,
    p.brand,
    p.product_category,
    p.product_type,
    p.manufacturer_id,
    d.age,
    d.income,
    COALESCE(d.home_ownership, 'Unknown') AS home_ownership,
    COALESCE(d.marital_status, 'Unknown') AS marital_status,
    d.household_size,
    d.household_comp,
    COALESCE(d.kids_count, 'Unknown') AS kids_count
FROM transactions_sample t
LEFT JOIN promotions_sample pr
    ON t.product_id = pr.product_id
   AND t.store_id = pr.store_id
   AND t.week = pr.week
LEFT JOIN products p
    ON t.product_id = p.product_id
LEFT JOIN demographics d
    ON t.household_id = d.household_id;
```

Initiative-level performance comparison

The first analytical query compares promotion placements, campaign windows, and redeemed coupons using one KPI framework. It aggregates households reached, trips, sales, units, average basket value, and purchase frequency, then ranks each initiative inside its own initiative type to highlight which commercial levers generated the strongest observed outcomes.

SQL QUERY · INITIATIVE PERFORMANCE

```
WITH promo_perf AS (
    SELECT
        'Promotion placement' AS initiative_type,
        CONCAT(display_location, ' | ', mailer_location) AS initiative_name,
        COUNT(DISTINCT household_id) AS households,
        COUNT(DISTINCT basket_id) AS trips,
        SUM(sales_value) AS net_sales,
        SUM(quantity) AS units,
        SUM(sales_value) / NULLIF(COUNT(DISTINCT basket_id), 0) AS avg_basket_value,
        COUNT(DISTINCT basket_id)::numeric / NULLIF(COUNT(DISTINCT household_id), 0) AS purchase_frequency
    FROM vw_txn_enriched
    WHERE promo_exposed = 1
    GROUP BY 1, 2
),
campaign_perf AS (
    SELECT
        'Campaign' AS initiative_type,
        CONCAT('Campaign ', c.campaign_id, ' | ', cd.campaign_type) AS initiative_name,
        COUNT(DISTINCT t.household_id) AS households,
        COUNT(DISTINCT t.basket_id) AS trips,
        SUM(t.sales_value) AS net_sales,
        SUM(t.quantity) AS units,
        SUM(t.sales_value) / NULLIF(COUNT(DISTINCT t.basket_id), 0) AS avg_basket_value,
        COUNT(DISTINCT t.basket_id)::numeric / NULLIF(COUNT(DISTINCT t.household_id), 0) AS
purchase_frequency
    FROM vw_txn_enriched t
    JOIN campaigns c
        ON t.household_id = c.household_id
    JOIN campaign_descriptions cd
        ON c.campaign_id = cd.campaign_id
       AND t.transaction_date BETWEEN cd.start_date AND cd.end_date
```

```
    GROUP BY 1, 2
),
coupon_perf AS (
    SELECT
        'Redeemed coupon' AS initiative_type,
        CONCAT('Coupon ', cp.coupon_upc, ' | Campaign ', cp.campaign_id) AS initiative_name,
        COUNT(DISTINCT t.household_id) AS households,
        COUNT(DISTINCT t.basket_id) AS trips,
        SUM(t.sales_value) AS net_sales,
        SUM(t.quantity) AS units,
        SUM(t.sales_value) / NULLIF(COUNT(DISTINCT t.basket_id), 0) AS avg_basket_value,
        COUNT(DISTINCT t.basket_id)::numeric / NULLIF(COUNT(DISTINCT t.household_id), 0) AS
purchase_frequency
    FROM coupon_redemptions cr
    JOIN coupons cp
        ON cr.campaign_id = cp.campaign_id
       AND cr.coupon_upc = cp.coupon_upc
    JOIN vw_txn_enriched t
        ON t.household_id = cr.household_id
       AND t.product_id = cp.product_id
       AND t.transaction_date = cr.redemption_date
    GROUP BY 1, 2
),
all_perf AS (
    SELECT * FROM promo_perf
    UNION ALL
    SELECT * FROM campaign_perf
    UNION ALL
    SELECT * FROM coupon_perf
)
SELECT
    initiative_type,
    initiative_name,
    households,
    trips,
    ROUND(net_sales::numeric, 2) AS net_sales,
    ROUND(units::numeric, 2) AS units,
    ROUND(avg_basket_value::numeric, 2) AS avg_basket_value,
    ROUND(purchase_frequency::numeric, 2) AS purchase_frequency,
    DENSE_RANK() OVER (
        PARTITION BY initiative_type
        ORDER BY net_sales DESC, purchase_frequency DESC, avg_basket_value DESC
    ) AS performance_rank
FROM all_perf
ORDER BY initiative_type, performance_rank;
```

Category-level promotional responsiveness

This query evaluates category responsiveness by comparing promoted and non-promoted performance within the consolidated analytical view. The output combines support metrics such as trips and households with performance measures such as sales per trip, sales lift, and unit lift. Because promotional support is sparse in several categories, the query is intended for directional interpretation rather than definitive optimization.

SQL QUERY · CATEGORY RESPONSIVENESS

```
WITH grain_perf AS (
    SELECT
        product_category,
        COUNT(DISTINCT CONCAT_WS('-', household_id, store_id, basket_id, week))
            FILTER (WHERE promo_exposed = 1) AS promo_trips,
        COUNT(DISTINCT CONCAT_WS('-', household_id, store_id, basket_id, week))
            FILTER (WHERE promo_exposed = 0) AS base_trips,
        COUNT(DISTINCT household_id) FILTER (WHERE promo_exposed = 1) AS promo_households,
        COUNT(DISTINCT household_id) FILTER (WHERE promo_exposed = 0) AS base_households,
        SUM(sales_value) FILTER (WHERE promo_exposed = 1) AS promo_sales,
        SUM(sales_value) FILTER (WHERE promo_exposed = 0) AS base_sales,
        SUM(quantity) FILTER (WHERE promo_exposed = 1) AS promo_units,
        SUM(quantity) FILTER (WHERE promo_exposed = 0) AS base_units
    FROM vw_txn_enriched
    GROUP BY 1
),
scored AS (
    SELECT
```

```
        product_category,
        promo_trips,
        base_trips,
        promo_households,
        base_households,
        promo_sales,
        base_sales,
        promo_units,
        base_units,
        promo_sales / NULLIF(promo_trips, 0) AS promo_sales_per_trip,
        base_sales / NULLIF(base_trips, 0) AS base_sales_per_trip,
        (promo_sales / NULLIF(promo_households, 0))
            / NULLIF(base_sales / NULLIF(base_households, 0), 0) - 1 AS sales_lift_pct,
        (promo_units / NULLIF(promo_households, 0))
            / NULLIF(base_units / NULLIF(base_households, 0), 0) - 1 AS unit_lift_pct
    FROM grain_perf
    WHERE promo_trips >= 5
      AND base_trips >= 20
      AND promo_households >= 3
)
SELECT
    product_category,
    promo_trips,
    base_trips,
    promo_households,
    base_households,
    ROUND(promo_sales::numeric, 2) AS promo_sales,
    ROUND(base_sales::numeric, 2) AS base_sales,
    ROUND(promo_sales_per_trip::numeric, 2) AS promo_sales_per_trip,
    ROUND(base_sales_per_trip::numeric, 2) AS base_sales_per_trip,
    ROUND(sales_lift_pct::numeric, 4) AS sales_lift_pct,
    ROUND(unit_lift_pct::numeric, 4) AS unit_lift_pct,
    DENSE_RANK() OVER (
        ORDER BY sales_lift_pct DESC, unit_lift_pct DESC, promo_sales DESC
    ) AS responsiveness_rank
FROM scored
ORDER BY responsiveness_rank
LIMIT 25;
```

Household-segment response analysis

This query calculates household-level change during campaign windows relative to a matched pre-campaign baseline and then aggregates those response measures by demographic segment. The approach allows sales, trip, and unit lift to be interpreted in a way that is more behaviorally meaningful than a single exposed-versus-non-exposed comparison.

SQL QUERY · HOUSEHOLD SEGMENTATION

```
WITH campaign_windows AS (
    SELECT
        c.household_id,
        c.campaign_id,
        cd.start_date::date AS start_date,
        cd.end_date::date AS end_date,
        (cd.end_date::date - cd.start_date::date + 1) AS window_days,
        GREATEST((cd.end_date::date - cd.start_date::date + 1) / 7.0, 1.0) AS window_weeks
    FROM campaigns c
    JOIN campaign_descriptions cd
        ON c.campaign_id = cd.campaign_id
),
household_campaign_perf AS (
    SELECT
        cw.household_id,
        cw.campaign_id,
        COALESCE(
            SUM(t.sales_value) FILTER (
                WHERE t.transaction_date BETWEEN cw.start_date AND cw.end_date
            ) / NULLIF(cw.window_weeks, 0), 0
        ) AS during_sales_per_week,
        COALESCE(
            COUNT(DISTINCT CONCAT_WS('-', t.household_id, t.store_id, t.basket_id, t.week)) FILTER (
                WHERE t.transaction_date BETWEEN cw.start_date AND cw.end_date
            ) / NULLIF(cw.window_weeks, 0), 0
```

```
        ) AS during_trips_per_week,
        COALESCE(
            SUM(t.quantity) FILTER (
                WHERE t.transaction_date BETWEEN cw.start_date AND cw.end_date
            ) / NULLIF(cw.window_weeks, 0), 0
        ) AS during_units_per_week,
        COALESCE(
            SUM(t.sales_value) FILTER (
                WHERE t.transaction_date BETWEEN (cw.start_date - cw.window_days) AND (cw.start_date - 1)
            ) / NULLIF(cw.window_weeks, 0), 0
        ) AS pre_sales_per_week,
        COALESCE(
            COUNT(DISTINCT CONCAT_WS('-', t.household_id, t.store_id, t.basket_id, t.week)) FILTER (
                WHERE t.transaction_date BETWEEN (cw.start_date - cw.window_days) AND (cw.start_date - 1)
            ) / NULLIF(cw.window_weeks, 0), 0
        ) AS pre_trips_per_week,
        COALESCE(
            SUM(t.quantity) FILTER (
                WHERE t.transaction_date BETWEEN (cw.start_date - cw.window_days) AND (cw.start_date - 1)
            ) / NULLIF(cw.window_weeks, 0), 0
        ) AS pre_units_per_week
    FROM campaign_windows cw
    LEFT JOIN vw_txn_enriched t
        ON cw.household_id = t.household_id
    GROUP BY
        cw.household_id,
        cw.campaign_id,
        cw.window_days,
        cw.window_weeks,
        cw.start_date,
        cw.end_date
),
household_response AS (
    SELECT
        household_id,
        COUNT(DISTINCT campaign_id) AS campaigns_received,
        AVG(during_sales_per_week) AS during_sales_per_week,
        AVG(pre_sales_per_week) AS pre_sales_per_week,
        AVG(during_trips_per_week) AS during_trips_per_week,
        AVG(pre_trips_per_week) AS pre_trips_per_week,
        AVG(during_units_per_week) AS during_units_per_week,
        AVG(pre_units_per_week) AS pre_units_per_week,
        AVG(
            CASE
                WHEN during_sales_per_week > pre_sales_per_week THEN 1.0
                ELSE 0.0
            END
        ) AS responder_rate
    FROM household_campaign_perf
    GROUP BY household_id
),
segment_base AS (
    SELECT
        'income' AS segment_dimension,
        d.income AS segment_value,
        hr.household_id,
        hr.campaigns_received,
        hr.during_sales_per_week,
        hr.pre_sales_per_week,
        hr.during_trips_per_week,
        hr.pre_trips_per_week,
        hr.during_units_per_week,
        hr.pre_units_per_week,
        hr.responder_rate
    FROM household_response hr
    JOIN demographics d
        ON hr.household_id = d.household_id
    WHERE d.income IS NOT NULL

    UNION ALL

    SELECT
        'age' AS segment_dimension,
        d.age AS segment_value,
```

```
        hr.household_id,
        hr.campaigns_received,
        hr.during_sales_per_week,
        hr.pre_sales_per_week,
        hr.during_trips_per_week,
        hr.pre_trips_per_week,
        hr.during_units_per_week,
        hr.pre_units_per_week,
        hr.responder_rate
    FROM household_response hr
    JOIN demographics d
        ON hr.household_id = d.household_id
    WHERE d.age IS NOT NULL

    UNION ALL

    SELECT
        'household_comp' AS segment_dimension,
        d.household_comp AS segment_value,
        hr.household_id,
        hr.campaigns_received,
        hr.during_sales_per_week,
        hr.pre_sales_per_week,
        hr.during_trips_per_week,
        hr.pre_trips_per_week,
        hr.during_units_per_week,
        hr.pre_units_per_week,
        hr.responder_rate
    FROM household_response hr
    JOIN demographics d
        ON hr.household_id = d.household_id
    WHERE d.household_comp IS NOT NULL
)
SELECT
    segment_dimension,
    segment_value,
    COUNT(*) AS households,
    ROUND(AVG(campaigns_received)::numeric, 2) AS avg_campaigns_received,
    ROUND(AVG(during_sales_per_week)::numeric, 2) AS during_sales_per_hh_week,
    ROUND(AVG(pre_sales_per_week)::numeric, 2) AS pre_sales_per_hh_week,
    ROUND((AVG(during_sales_per_week) / NULLIF(AVG(pre_sales_per_week), 0) - 1)::numeric, 4) AS
sales_lift_pct,
    ROUND(AVG(during_trips_per_week)::numeric, 2) AS during_trips_per_hh_week,
    ROUND(AVG(pre_trips_per_week)::numeric, 2) AS pre_trips_per_hh_week,
    ROUND((AVG(during_trips_per_week) / NULLIF(AVG(pre_trips_per_week), 0) - 1)::numeric, 4) AS
trip_lift_pct,
    ROUND(AVG(during_units_per_week)::numeric, 2) AS during_units_per_hh_week,
    ROUND(AVG(pre_units_per_week)::numeric, 2) AS pre_units_per_hh_week,
    ROUND((AVG(during_units_per_week) / NULLIF(AVG(pre_units_per_week), 0) - 1)::numeric, 4) AS
unit_lift_pct,
    ROUND(AVG(responder_rate)::numeric, 4) AS responder_rate
FROM segment_base
GROUP BY 1, 2
HAVING COUNT(*) >= 10
ORDER BY segment_dimension, sales_lift_pct DESC;
```

Exposed versus non-exposed cohort comparison

The cohort comparison query classifies households according to whether they had any campaign participation, promotional exposure, or coupon redemption. Household-level total behavior is then summarized into average and median sales, trips, basket value, and units to provide a descriptive comparison of overall commercial profile across the two cohorts.

SQL QUERY · EXPOSURE COHORT COMPARISON

```
WITH all_households AS (
    SELECT DISTINCT household_id
    FROM transactions_sample
),
household_flags AS (
    SELECT
        h.household_id,
        CASE
```

```
            WHEN EXISTS (
                SELECT 1
                FROM campaigns c
                WHERE c.household_id = h.household_id
            )
            OR EXISTS (
                SELECT 1
                FROM vw_txn_enriched t
                WHERE t.household_id = h.household_id
                  AND t.promo_exposed = 1
            )
            OR EXISTS (
                SELECT 1
                FROM coupon_redemptions cr
                WHERE cr.household_id = h.household_id
            )
            THEN 'Exposed'
            ELSE 'Non-exposed'
        END AS exposure_cohort
    FROM all_households h
),
household_metrics AS (
    SELECT
        hf.exposure_cohort,
        t.household_id,
        SUM(t.sales_value) AS net_sales,
        SUM(t.quantity) AS units,
        COUNT(DISTINCT t.basket_id) AS trips,
        SUM(t.sales_value) / NULLIF(COUNT(DISTINCT t.basket_id), 0) AS avg_basket_value
    FROM household_flags hf
    JOIN transactions_sample t
        ON hf.household_id = t.household_id
    GROUP BY 1, 2
)
SELECT
    exposure_cohort,
    COUNT(*) AS households,
    ROUND(AVG(net_sales)::numeric, 2) AS avg_sales_per_household,
    ROUND(PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY net_sales)::numeric, 2) AS median_sales_per_household,
    ROUND(AVG(trips)::numeric, 2) AS avg_trips_per_household,
    ROUND(AVG(avg_basket_value)::numeric, 2) AS avg_basket_value,
    ROUND(AVG(units)::numeric, 2) AS avg_units_per_household
FROM household_metrics
GROUP BY 1
ORDER BY 1;
```

Campaign-level incremental performance

This query estimates campaign-level incrementality by comparing household behavior during campaign windows against an equally sized pre-period for the same households. The resulting campaign summary produces sales lift, trip lift, and responder rate so campaigns can be classified into stronger, mixed, or weaker directional performance patterns.

SQL QUERY · CAMPAIGN INCREMENTALITY

```
WITH campaign_windows AS (
    SELECT
        c.household_id,
        c.campaign_id,
        cd.campaign_type,
        cd.start_date::date AS start_date,
        cd.end_date::date AS end_date,
        (cd.end_date::date - cd.start_date::date + 1) AS window_days,
        GREATEST((cd.end_date::date - cd.start_date::date + 1) / 7.0, 1.0) AS window_weeks
    FROM campaigns c
    JOIN campaign_descriptions cd
        ON c.campaign_id = cd.campaign_id
),
household_campaign_perf AS (
    SELECT
        cw.campaign_id,
        cw.campaign_type,
        cw.household_id,
```

```
        COALESCE(
            SUM(t.sales_value) FILTER (
                WHERE t.transaction_date BETWEEN cw.start_date AND cw.end_date
            ) / NULLIF(cw.window_weeks, 0), 0
        ) AS during_sales_per_hh_week,
        COALESCE(
            COUNT(DISTINCT t.basket_id) FILTER (
                WHERE t.transaction_date BETWEEN cw.start_date AND cw.end_date
            ) / NULLIF(cw.window_weeks, 0), 0
        ) AS during_trips_per_hh_week,
        COALESCE(
            SUM(t.sales_value) FILTER (
                WHERE t.transaction_date BETWEEN (cw.start_date - cw.window_days) AND (cw.start_date - 1)
            ) / NULLIF(cw.window_weeks, 0), 0
        ) AS pre_sales_per_hh_week,
        COALESCE(
            COUNT(DISTINCT t.basket_id) FILTER (
                WHERE t.transaction_date BETWEEN (cw.start_date - cw.window_days) AND (cw.start_date - 1)
            ) / NULLIF(cw.window_weeks, 0), 0
        ) AS pre_trips_per_hh_week
    FROM campaign_windows cw
    LEFT JOIN vw_txn_enriched t
        ON cw.household_id = t.household_id
    GROUP BY
        cw.campaign_id,
        cw.campaign_type,
        cw.household_id,
        cw.window_days,
        cw.window_weeks,
        cw.start_date,
        cw.end_date
),
campaign_summary AS (
    SELECT
        campaign_id,
        campaign_type,
        COUNT(*) AS assigned_households,
        AVG(during_sales_per_hh_week) AS during_sales_per_hh_week,
        AVG(pre_sales_per_hh_week) AS pre_sales_per_hh_week,
        AVG(during_trips_per_hh_week) AS during_trips_per_hh_week,
        AVG(pre_trips_per_hh_week) AS pre_trips_per_hh_week,
        AVG(
            CASE
                WHEN during_sales_per_hh_week > pre_sales_per_hh_week THEN 1.0
                ELSE 0.0
            END
        ) AS responder_rate
    FROM household_campaign_perf
    GROUP BY 1, 2
)
SELECT
    campaign_id,
    campaign_type,
    assigned_households,
    ROUND(during_sales_per_hh_week::numeric, 2) AS during_sales_per_hh_week,
    ROUND(pre_sales_per_hh_week::numeric, 2) AS pre_sales_per_hh_week,
    ROUND((during_sales_per_hh_week / NULLIF(pre_sales_per_hh_week, 0) - 1)::numeric, 4) AS sales_lift_pct,
    ROUND((during_trips_per_hh_week / NULLIF(pre_trips_per_hh_week, 0) - 1)::numeric, 4) AS trip_lift_pct,
    ROUND(responder_rate::numeric, 4) AS responder_rate,
    CASE
        WHEN (during_sales_per_hh_week / NULLIF(pre_sales_per_hh_week, 0) - 1) >= 0.10
             AND responder_rate >= 0.55
            THEN 'Meaningful incremental impact'
        WHEN (during_sales_per_hh_week / NULLIF(pre_sales_per_hh_week, 0) - 1) BETWEEN 0.00 AND 0.10
            THEN 'Moderate / mixed'
        ELSE 'Limited or inconsistent'
    END AS initiative_class
FROM campaign_summary
ORDER BY sales_lift_pct DESC, responder_rate DESC;
```

Category opportunity matrix

The opportunity matrix combines exposure rate and sales-per-trip lift to identify categories that warrant scale, protection, redesign, or selective monitoring. It translates analytical output into a simple decision-oriented structure that is easier to communicate to a commercial audience than a raw ranking table alone.

SQL QUERY · CATEGORY OPPORTUNITY MATRIX

```
WITH category_perf AS (
    SELECT
        department,
        product_category,
        COUNT(DISTINCT household_id) FILTER (WHERE promo_exposed = 1) AS exposed_households,
        COUNT(DISTINCT household_id) AS total_households,
        COUNT(DISTINCT basket_id) FILTER (WHERE promo_exposed = 1) AS promo_trips,
        COUNT(DISTINCT basket_id) FILTER (WHERE promo_exposed = 0) AS base_trips,
        SUM(sales_value) FILTER (WHERE promo_exposed = 1) AS promo_sales,
        SUM(sales_value) FILTER (WHERE promo_exposed = 0) AS base_sales
    FROM vw_txn_enriched
    GROUP BY 1, 2
),
scored AS (
    SELECT
        department,
        product_category,
        exposed_households::numeric / NULLIF(total_households, 0) AS exposure_rate,
        (promo_sales / NULLIF(promo_trips, 0))
            / NULLIF(base_sales / NULLIF(base_trips, 0), 0) - 1 AS sales_per_trip_lift_pct,
        PERCENT_RANK() OVER (
            ORDER BY exposed_households::numeric / NULLIF(total_households, 0)
        ) AS exposure_pctile,
        PERCENT_RANK() OVER (
            ORDER BY (promo_sales / NULLIF(promo_trips, 0))
                     / NULLIF(base_sales / NULLIF(base_trips, 0), 0) - 1
        ) AS lift_pctile
    FROM category_perf
    WHERE promo_trips >= 5
      AND base_trips >= 10
)
SELECT
    department,
    product_category,
    ROUND(exposure_rate::numeric, 4) AS exposure_rate,
    ROUND(sales_per_trip_lift_pct::numeric, 4) AS sales_per_trip_lift_pct,
    CASE
        WHEN lift_pctile >= 0.75 AND exposure_pctile < 0.50 THEN 'Scale investment'
        WHEN lift_pctile >= 0.75 AND exposure_pctile >= 0.50 THEN 'Protect and optimize'
        WHEN lift_pctile < 0.50 AND exposure_pctile >= 0.75 THEN 'Reduce or redesign'
        ELSE 'Selective test / monitor'
    END AS recommended_action
FROM scored
ORDER BY sales_per_trip_lift_pct DESC, exposure_rate ASC;
```

Coupon redemption and follow-up behavior

The final query links eligible households, campaign windows, coupon redemptions, and same-product post-redemption transactions. It estimates redemption rate, repeat redemption rate, redemption lag, sales per redeemer, and trips per redeemer to provide a directional view of how redeemed coupons relate to subsequent product purchases during the campaign period.

SQL QUERY · COUPON BEHAVIOR

```
WITH eligible AS (
    SELECT DISTINCT
        c.household_id,
        cp.campaign_id,
        cp.coupon_upc,
        cp.product_id
    FROM campaigns c
    JOIN coupons cp
        ON c.campaign_id = cp.campaign_id
),
```

```
campaign_window AS (
    SELECT
        campaign_id,
        MIN(start_date) AS start_date,
        MAX(end_date) AS end_date
    FROM campaign_descriptions
    GROUP BY 1
),
redeemed AS (
    SELECT
        household_id,
        campaign_id,
        coupon_upc,
        MIN(redemption_date) AS first_redemption_date,
        COUNT(*) AS redemption_events
    FROM coupon_redemptions
    GROUP BY 1, 2, 3
),
base AS (
    SELECT
        e.household_id,
        e.campaign_id,
        e.coupon_upc,
        e.product_id,
        p.product_category,
        cw.start_date,
        cw.end_date,
        r.first_redemption_date,
        r.redemption_events
    FROM eligible e
    JOIN products p
        ON e.product_id = p.product_id
    JOIN campaign_window cw
        ON e.campaign_id = cw.campaign_id
    LEFT JOIN redeemed r
        ON e.household_id = r.household_id
       AND e.campaign_id = r.campaign_id
       AND e.coupon_upc = r.coupon_upc
),
household_stats AS (
    SELECT
        campaign_id,
        product_category,
        COUNT(DISTINCT household_id) AS eligible_households,
        COUNT(DISTINCT household_id) FILTER (
            WHERE first_redemption_date IS NOT NULL
        ) AS redeemed_households,
        COUNT(DISTINCT household_id) FILTER (
            WHERE redemption_events > 1
        ) AS repeat_redeemers,
        AVG((first_redemption_date::date - start_date::date)::numeric) FILTER (
            WHERE first_redemption_date IS NOT NULL
        ) AS avg_redemption_lag_days
    FROM base
    GROUP BY 1, 2
),
post_redemption_txn AS (
    SELECT
        b.campaign_id,
        b.product_category,
        SUM(t.sales_value) AS sales_after_redemption,
        COUNT(DISTINCT t.basket_id) AS trips_after_redemption
    FROM base b
    JOIN transactions_sample t
        ON t.household_id = b.household_id
       AND t.product_id = b.product_id
       AND b.first_redemption_date IS NOT NULL
       AND t.transaction_timestamp >= b.first_redemption_date::timestamp
       AND t.transaction_timestamp < (b.end_date + INTERVAL '1 day')
    GROUP BY 1, 2
)
SELECT
    hs.campaign_id,
    hs.product_category,
```

```
    hs.eligible_households,
    hs.redeemed_households,
    ROUND(hs.redeemed_households::numeric / NULLIF(hs.eligible_households, 0), 4) AS redemption_rate,
    ROUND(hs.repeat_redeemers::numeric / NULLIF(hs.redeemed_households, 0), 4) AS repeat_redemption_rate,
    ROUND(hs.avg_redemption_lag_days, 2) AS avg_redemption_lag_days,
    ROUND(COALESCE(pr.sales_after_redemption, 0)::numeric / NULLIF(hs.redeemed_households, 0), 2) AS
sales_per_redeemer,
    ROUND(COALESCE(pr.trips_after_redemption, 0)::numeric / NULLIF(hs.redeemed_households, 0), 2) AS
trips_per_redeemer
FROM household_stats hs
LEFT JOIN post_redemption_txn pr
    ON hs.campaign_id = pr.campaign_id
   AND hs.product_category = pr.product_category
ORDER BY redemption_rate DESC, sales_per_redeemer DESC;
```

Implementation notes

- Connection credentials were intentionally redacted in this appendix to preserve a professional and portable representation of the loading workflow.
- The analytical layer relies on sampled promotion and transaction tables for most SQL development, so category-level and exposure-based outputs should be interpreted directionally where support is thin.
- Indexes were added on the most frequent join and filter keys to make iterative query development more efficient inside PostgreSQL.
- The final SQL design prioritizes reusability: the consolidated view standardizes repeated enrichment work so downstream queries remain shorter, more readable, and easier to audit.